RÉSUMÉ : SÉCURITÉ AVEC JWT

Partie 1/2 — Le jeton JWT (JSON Web Token)

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 4 · 2026-2027

1 QU'EST-CE QU'UN JWT ?

JWT (JSON Web Token) : standard défini par la **RFC 7519** pour transmettre entre applications des informations (des *claims*) sous forme d'un objet JSON **signé**, compact et autonome.

Compact
petite taille → transmis rapidement, en général dans l'en-tête HTTP **Authorization**.

Autonome
contient les infos utiles sur l'utilisateur → évite d'interroger la base à chaque requête.

Attention : la signature garantit l'**intégrité** et l'**authenticité**, pas la confidentialité. Un JWT signé est lisible par tous.

2 ANATOMIE D'UN JETON

Trois parties encodées en **Base64URL**, séparées par un point : `xxx.yyy.zzz`

```
eyJhbGciOiJIUzI1NiJ9.
eyJzdWIiOiJJbmVzIn0.
TJVA95OrM7E2cBab30RM
```

HEADER	PAYLOAD	SIGNATURE
type du jeton + algorithme de signature	les données transportées (claims)	garantit intégrité et authenticité

3 POURQUOI JWT ? SESSIONS VS JETONS

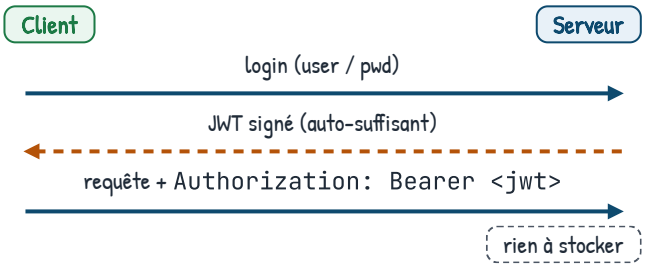
Authentification par SESSION (avec état)



Le serveur doit retrouver l'état à chaque requête.

× état côté serveur · × scalabilité limitée · × partage entre services difficile

Authentification par JWT (sans état)



Le serveur vérifie seulement la signature.

✓ sans état, scalable · ✓ facile à partager entre microservices · × révocation moins simple

4 LE HEADER

```
{
 "alg": "HS256",
 "typ": "JWT"
}
```

alg : algorithme de signature.
typ : type du jeton (facultatif).

HMAC	HS256/384/512 — **symétrique** : un secret partagé
RSA	RS256/384/512 — **asymétrique** : clé privée signe, clé publique vérifie
RSA-PSS	PS256/384/512 — asymétrique
ECDSA	ES256/384/512 — asymétrique, clés plus courtes

5 LA SIGNATURE

Calculée sur le header et le payload encodés, avec la clé (ici un secret HMAC) :

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret)
```

Elle permet de vérifier que l'émetteur est bien celui qu'il prétend être (**authenticité**) et que le jeton n'a pas été modifié (**intégrité**).

6 LE PAYLOAD : LES CLAIMS

Claims enregistrés (RFC 7519, tous facultatifs) :

`iss`	émetteur (issuer) du jeton
`sub`	sujet : l'utilisateur concerné
`aud`	public cible (audience)
`exp`	date d'expiration
`nbf`	« not before » : pas valide avant cette date
`iat`	« issued at » : date de création
`jti`	identifiant unique du jeton

Claims privés : définis par l'application (ex. `roles`).

```
{
 "sub": "Ines",
 "iat": 1700000000,
 "exp": 1700003600,
 "jti": "idr56543ftu8909876",
 "roles": ["admin", "author"]
}
```

Dates en secondes depuis le 1/1/1970 : ici, exp = iat + 1 h.

Base64URL = encodage, pas chiffrement : jamais de données sensibles (mot de passe…) dans le payload.

RÉSUMÉ : SÉCURITÉ AVEC JWT

Partie 2/2 — Utiliser JWT en toute sécurité

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 4 · 2026-2027

7 COMMENT UTILISER JWT



Le jeton est envoyé à **chaque requête** vers une ressource protégée, dans l'en-tête **Authorization** avec le schéma **Bearer**.

8 OÙ STOCKER LE JETON (CLIENT) ?

✕ **localStorage** : lisible par JavaScript → volé en cas d'attaque **XSS**.

✓ **Cookie httpOnly + Secure + SameSite** : inaccessible au JavaScript (prévoir aussi une protection CSRF).

✓ **Access token en mémoire** + refresh token en cookie httpOnly.

→ Toujours sur **HTTPS** : JWT ne remplace pas le chiffrement du transport.

9 VALIDER UN JWT CÔTÉ SERVEUR

Décoder ≠ valider : sans vérifier la signature, n'importe qui peut forger un payload.

1 Imposer l'**algorithme attendu** (jamais celui choisi par le jeton)

2 Vérifier la **signature** avec la bonne clé

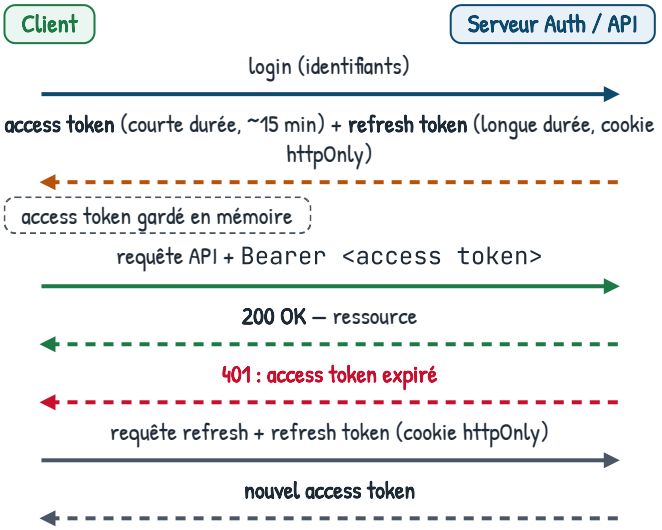
3 Vérifier **exp** et **nbf** (dates de validité)

4 Vérifier **iss** (émetteur) et **aud** (destinataire)

5 Puis contrôler les **droits** (roles) sur la ressource demandée

→ Jeton absent, invalide ou expiré : **401** · droits insuffisants : **403**.

10 ACCESS TOKEN VS REFRESH TOKEN



Si le refresh token est aussi expiré ou révoqué, le client doit se ré-authentifier.

Access token	Refresh token
courte durée ; envoyé à chaque requête API	longue durée ; sert seulement à obtenir un nouvel access token

11 VULNÉRABILITÉS ET PARADES

Risque	Parade
« alg: none » : le header est modifié pour désactiver la vérification	Imposer l'algorithme attendu côté serveur
Confusion RS256 → HS256 : la clé publique RSA sert de secret HMAC	Valider explicitement l'algorithme et la clé
Jeton lisible : signé, pas chiffré	Aucune donnée sensible dans le payload
Pas de révocation native : un jeton volé reste valide jusqu'à exp	Durée de vie courte + refresh tokens, liste noire (jti)
Claims non vérifiés	Toujours vérifier exp, nbf, iss, aud

12 SYNTHÈSE : AVANTAGES ET LIMITES

Avantages

- ✓ **Sans état** : scalable, idéal pour microservices et API distribuées
- ✓ **Compact et auto-porteur** : les infos utiles sont dans le jeton
- ✓ **Interopérable** : standard ouvert (RFC 7519), supporté partout

Limites

- ✕ Pas de **révocation** native avant expiration
- ✕ **Non chiffré** par défaut : jamais de données sensibles
- ✕ La **taille** du jeton grossit avec le nombre de claims

POINTS À RETENIR

Un JWT = **header.payload.signature** en Base64URL ; il est **signé**, pas chiffré.

Le client l'envoie dans **Authorization: Bearer** ; le serveur **valide** signature et claims à chaque requête.

Access token **court**, refresh token **long** ; toujours sur **HTTPS**, jamais en localStorage.